The League — how our bot trains itself

Patient Wolves, Battlecode 2026a, branch `league`

October 2026

Quick start

Open PowerShell in the repo folder and activate the environment once per window:

```
.\.venv\Scripts\Activate.ps1
```

1. Start training (runs in the background until you stop it):

```
scripts\train.ps1 start                          # a normal PC
scripts\train.ps1 start --lead                   # the ONE lead PC, no uploads
scripts\train.ps1 start --lead --auto-submit     # the ONE lead PC, may upload
scripts\train.ps1 stop                           # stop it
```

Only one PC in the team may use `--lead`. `--auto-submit` needs `--lead` and a logged-in cambc (`cambc login`). The upload rules are in §5.

2. See how the latest model is doing:

```
scripts\train.ps1 status        # version, win rate per top team, losses
scripts\train.ps1 logs          # live log (Ctrl+C to leave)
scripts\train.ps1 bench         # new test vs every mimic, every map
scripts\train.ps1 review        # full report: league\review\<date>.md
```

3. Watch it play (opens the official viewer in your browser):

```
scripts\train.ps1 replay                    # our worst team + map
scripts\train.ps1 replay nyx lanes          # vs Nyx's mimic on lanes
scripts\train.ps1 replay nyx lanes --evolved   # vs the harder Nyx
```

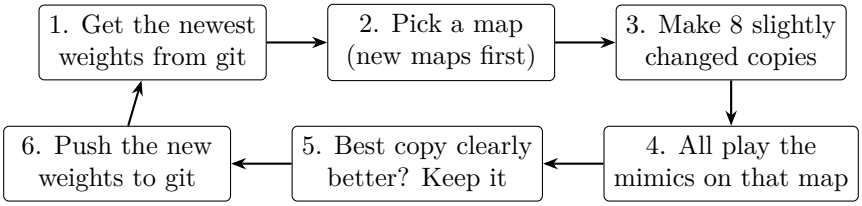
It plays two games (one from each side) and prints win/loss and gems. The viewer opens the first game, and the command for the second is printed. Stop the viewer with Ctrl+C. Real ladder games: `cambc matches download <matchId> -o x.repl26a`, then `cambc view x.repl26a`.

1 The idea in five sentences

1. We copy the 10 best teams on the ladder into practice bots, called **mimics**.
2. Our bot plays thousands of games against these mimics on every map.
3. After each batch of games we try small changes to our bot's numbers, and keep the changes that win more.
4. When we lose on the real ladder, the bot trains on that map first, and a better version can be uploaded.
5. Anything the training cannot fix is written in a report for a human to look at.

Every teammate's PC runs the same program (`harness/league.py`). They share the results through git.

2 One training round



A round takes about 4 minutes. Every third round, we train one of the *mimics* instead of our bot (see §4).

Which map gets trained. In this order:

1. **Maps that were never trained** (new maps, or old ones still on default numbers). Each gets 4 rounds first, and maps we lost on the ladder go first.
2. **The oldest ladder loss**, unless its map already had 6 rounds since that loss. After that it goes to the human review.
3. **The map we do worst on** in the latest test, or simply the next map in turn.

Every map in the platform's pool always has an entry: a new map starts with the default numbers until training improves them. `train status` checks that the counts agree (*fingerprint 21/21, geometry 21/21, in pack 21/21, trained N/21*). It also lists each untrained map with its rounds so far, e.g. *hooks 2/4*. A map without a fingerprint is never trained, because the bot couldn't load its numbers, and `status` reports it as OUT OF SYNC.

3 What the bot learns

Code vs numbers

Our bot is hand-written Rust code. The code decides *how* to do things: find a path, attack, buy an item, deliver gems. Many of its decisions compare options with a formula, and those formulas contain numbers. Training never touches the code. It only changes these **102 numbers**.

Each map has its own set of 102 numbers. On a small map with camps close together, walking distance matters less than on a big map. On a map where fights happen early, retreating sooner may be better. A map we never trained on uses the default numbers in `bots/main/src/learned.rs`.

The numbers come in five groups.

1. W — 18 dials on our own settings

While writing the bot we picked many settings by hand, such as "retreat at 35% HP". Each W number *multiplies* one such setting. 1.0 means "as we wrote it", 1.2 means 20% more, and 0.8 means 20% less. For example, if the written retreat point is 35% HP, a `retreat_hp` of 1.2 makes the bot retreat at 42%. Allowed range: 0.2 to 5.

Dial	What turning it up does
`gem`, `gold`	Camps with more gems / gold look more attractive.
`dist`, `opening_dist`	Walking distance counts more, so near camps win (`opening_` only at game start).
`clear`	Time needed to kill a camp counts more, so quick camps win.
`strength_discount`	Strong (dangerous) camps look less attractive.
`reward`, `shared_small`	Bonus for camps that drop items; how much a small camp shared with allies is worth.
`hysteresis`	Stick with the current target instead of switching back and forth.
`en_detour`, `en_clear`	Allow longer detours and harder camps when clearing "on the way".
`fight_margin`	Need a bigger advantage before starting a fight (more careful).
`retreat_hp`	Retreat earlier (at higher HP).
`aggression`	Chase and attack enemies more.
`recall_gold`	Carry more gold before going home to shop.
`tribute_min`, `tribute_max`	Carry more gems before walking to the altar to deliver them.
`assist_hp`	Need more HP before running to help a teammate.

2. TARGET — 10 weights for "which camp next?"

Every time a champion picks its next camp, it gives each camp a score from the dials above. The TARGET weights then adjust that score with 10 facts about the camp:

gems in it • gold in it • distance • time to clear • time until it respawns • how many others are going for it • damage we would take • how full our bag is • our emblem count • the game round

Each weight is a number from −3 to +3, and 0 means "ignore this fact". A positive weight on *round* makes far camps more attractive late in the game. A negative weight on *damage* avoids camps that hurt. The camp's score is multiplied by $e^{\text{sum of weight}\times\text{fact}}$.

3. FIGHT — 9 weights for "is this fight good?"

Before fighting, the bot estimates its advantage. The FIGHT weights adjust that estimate with: our HP, enemy HP, allies nearby, enemies nearby, whether we are ranged, whether they are ranged, how full our bag is (a full bag means more to lose), and how far we are from our spawn, plus a constant. Range −3 to +3, and 0 = off. A negative weight on *bag* makes a champion carrying many gems avoid fights.

4. U — 60 weights for "what do I do right now?"

Every turn, a small decision maker scores 6 possible actions:

keep going • step back • support an ally • fight • walk home • recall (teleport home)

using 10 facts: a constant, our HP, damage per turn from camps, damage per turn from enemies, enemies nearby, allies nearby, an ally in trouble, bag size, distance to spawn, and enemy ranged. Each action has one weight per fact, so $6 \times 10 = 60$ numbers. The highest score wins. All zeros means "do what the hand-written rule says". Range −4 to +4.

5. X — 4 extras, and HOT

`tavi_respec` (> 0.5 = Tavi re-buys its stats mid-game). `arbiter_fuel_floor`: the U decision maker only runs when at least this much CPU fuel (25k–60k) is left in the turn, so we never freeze.

Two slots are unused. **HOT** (1 number) avoids places we died before; it is fixed at 0 because it made us worse.

Not learned

These are fixed code or fixed per-map plans, and only a human can change them:

- pathfinding, and how attacks and spells are aimed;
- the draft (which champions we pick);
- emblem build order, and the shopping list;
- which map features the bot can "see". If an important fact is missing from the lists above, no amount of training can use it. That is the "model can't express it" case in §6.

See all 102 names and their current defaults: `python harness\params.py --list`.

How training changes the numbers

One round works on one map:

1. Take the current best 102 numbers for that map.
2. Make 8 copies, each nudged by a small random amount. They come in 4 mirrored pairs: one copy goes + and its partner goes −, which makes the comparison fairer.
3. Every copy and the current best play the same opponents (4 mimics + 1 anchor bot) on that map, from both sides. That is about 90 games, run in parallel.
4. If the best copy clearly wins (see below), it becomes the new best for that map.
5. The nudge size adapts: after a success it grows by 15% (explore further), after a failure it shrinks by 15% (fine-tune).

How a copy is judged

Each bot gets a score over its games on that map:

$$\text{score} = \text{win rate} + 0.25 \times \text{gem lead} - 0.01 \times \text{average deaths}$$

Gem lead is (our gems − their gems)/(our gems + their gems), from −1 to +1. So a close loss still scores better than a crushing loss, and dying a lot costs a little.

Example. The current bot wins 6 of 10 games (0.6), has an average gem lead of +0.1, and dies 5 times a game: $0.6 + 0.025 - 0.05 = 0.575$. A copy that wins 7 of 10 with lead +0.2 and 4 deaths scores $0.7 + 0.05 - 0.04 = 0.71$. That is 0.135 better, so it passes the first test.

A copy replaces the current best only if **all four** hold:

(a) its score is at least **0.04** higher on the same games (small differences are luck);
(b) it beats the current best **head-to-head** on that map (at least 50%);
(c) it also scores at least as well against **3 other opponents** that it wasn't trained against (so it didn't just learn one opponent's habits);
(d) it runs out of CPU fuel no more than twice as often (a frozen champion loses games).

Then the new numbers are saved, a full test against every mimic on every map is run, and the result is pushed so all PCs use it. Games are deterministic: the same bots on the same map always play the same game. Any difference between two copies therefore comes from their numbers, not from luck in the game itself.

4 Who we train against

Faithful mimic (`mimic-nyx`)	A copy of a real team. From their recent ladder games we copy their champion picks, emblem builds, first camps on each map, and how often they deliver gems and shop. Their fighting can't be copied (the game data doesn't show it), so a mimic fights like our own bot. Never changed by training, so "our win rate vs Nyx" stays an honest number.
Evolved mimic (`mimx-nyx`)	The same copy, but its numbers are trained to *beat us*. Its picks, builds, first camps and delivery habits stay locked to the real team. If the real teams stop updating their bots, these keep getting harder, so our training doesn't stall. If the real team changes its bot, its evolved mimic starts over.
Anchor bots (`claude-*`, `exploit-*`, `baseline`)	Hand-written bots that attack known weak spots (for example killing our carrier near the altar). They stop old bugs coming back.

We pick opponents that beat us more often than ones we already beat.

5 When a new version goes on the ladder

Only one PC, the **lead**, talks to the platform. If it runs with `--auto-submit`, it uploads the newest trained version when **all** of these are true:

- the trained version is newer than the one on the ladder;
- at least 6 hours have passed since the last upload;
- we lost a ladder game since then, **or** the new version wins at least 5% more against the mimics.

If the new version then does more than 10% worse on the ladder (after 10 ranked games each), the previous version is put back.

6 What humans do

Every few hours the lead writes `league/review/<date>.md`. Read it, and write your thoughts in its *Reviewer notes* section. The program keeps that section.

The "gap" column compares our win rate against a team's mimic with our real win rate against that team. A big gap means the mimic is too easy, because the real team does something we didn't copy. Watch their replays.

Losses training couldn't fix (after 6 tries on that map) are listed with a hint:

- *"mimic can't express it"*: we beat their mimic but lost to the real team. Fix the mimic.
- *"model can't express it"*: we keep losing even to the mimic. None of the 102 numbers can fix it, so the bot needs a new feature in the code. Do it on a `feat/` branch, test it, and merge it into `league`. Every PC picks it up on its next round.

7 Maps

The lead follows the platform's current map pool (21 maps in October 2026). New maps are downloaded into `maps/`, and one test game per map records its ID and layout. Maps that leave the pool move to `maps/retired/` and are no longer trained on. Losses on a map we didn't have wait until it arrives.

8 The files that matter

`league/champion.json`	Our best numbers for each map, and the history of improvements. Shared by all PCs.
`bots/main/src/pack/pack.bin`	The same numbers in the format the bot reads. Rebuilt automatically.
`league/mimics/<team>.json`	The evolved mimics' numbers.
`league/ledger.json`	Ladder data: top 10, our games, lost games, uploads. Written by the lead.
`data/mimic/`	The copied teams.
`league/review/`	Reports for humans.
`runs/league/`	This PC's logs and scratch files. Safe to delete.

Never edit the `league/*.json` files or `pack.bin` by hand. If two PCs improve the bot at the same time, the program merges their work. On the same map, a head-to-head game decides which change wins.

9 Commands

Set up a new PC (needs cambc 3.0.1 or newer, the same engine as the ladder):

```
git clone -b league https://github.com/Ahmed-Abdul-Rahim/battlecode-2026a.git
cd battlecode-2026a
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1     (Linux/macOS: bash scripts/
    setup.sh)
.\.venv\Scripts\Activate.ps1
cambc login                       (lead PC only)
```

Run it (Linux/macOS: `scripts/train.sh` with the same words):

```
scripts\train.ps1 start                          normal PC: train and share
scripts\train.ps1 start --lead                   the one lead PC: also reads the ladder
scripts\train.ps1 start --lead --auto-submit     lead that may also upload (only one PC!)
scripts\train.ps1 status                         what is happening
scripts\train.ps1 logs                           live log
scripts\train.ps1 stop
scripts\train.ps1 once --no-push                 one round, no git (for testing)
scripts\train.ps1 sync | maps | bench | review   run one lead step now
scripts\train.ps1 replay [team] [map] [--evolved]   watch the latest model play
scripts\train.ps1 submit                         upload by hand (agree with the team first)
```

Look at games:

```
bctools run main mimic-nyx --maps 00lanes       play our bot vs a mimic
bctools gate main                               quick check vs the baseline bot
cambc view <replay>.repl26a                     watch a replay
cambc matches download <matchId> -o x.repl26a   get a ladder game from the report
```

Rebuild this PDF: `cd docs; latexmk -pdf league.tex; latexmk -c`

10 When something goes wrong

“already running”	Run `scripts\train.ps1 stop`, then start again.
“maps pull failed”	cambc is too old. Run `pip install -U "cambc>=3.0.1"`, then `cambc init --yes .`
“learned.rs and params.py disagree”	Someone changed the list of numbers. Update `harness/params.py` to match.
“build failed”	A broken code change was pulled. Fix it on a branch.
“merge needs a human”	Two people changed the same code. Fix the git conflict by hand.
“no mimics yet”	The lead hasn’t synced yet. Training uses the anchor bots meanwhile.
Results differ from the ladder	Check that your cambc is 3.0.1 or newer.
An evolved mimic looks broken	Delete `league/mimics/<team>.json` and push. It starts over.